

Lab 6 - Design of a Simple General-Purpose Processor

Giulia Monaco

COE328 Section 2

Dr. Geurkov

Y. Ahmadiadli

December 2, 2025

Table of contents

[1] Introduction.....	pg 3
[2] Components.....	pg 4
[3] Problem 1.....	pg 10
[4] Problem 2.....	pg 12
[5] Problem 3.....	pg 14
[6] Conclusion.....	pg 16

Introduction

The purpose of this lab was to design a General Processing Unit (GPU) in VHDL using an Arithmetic and Logic Unit (ALU), a 4:16 decoder, two latches, and a Final State Machine (FSM). The two latches act as storage units as they store the last 4 digits of the student number (A and B) that will be outputted to the ALU. They are dependent on the rising edge of the clock and can be reset at the user's discretion. The latch output will be fed into the ALU, which will perform the logic using the A and B values defined for each microcode. This microcode instruction comes from the 4:16 decoder. The FSM is responsible for controlling which microcode the decoder sends to the ALU as it is dependent on the current state of the system. Together, the decoder and FSM act as a control unit. The intersectionality and function of the components will further be discussed in the **Components** section. It must be noted that a block diagram will utilize all components described above in order for an output to be displayed on an FPGA board.

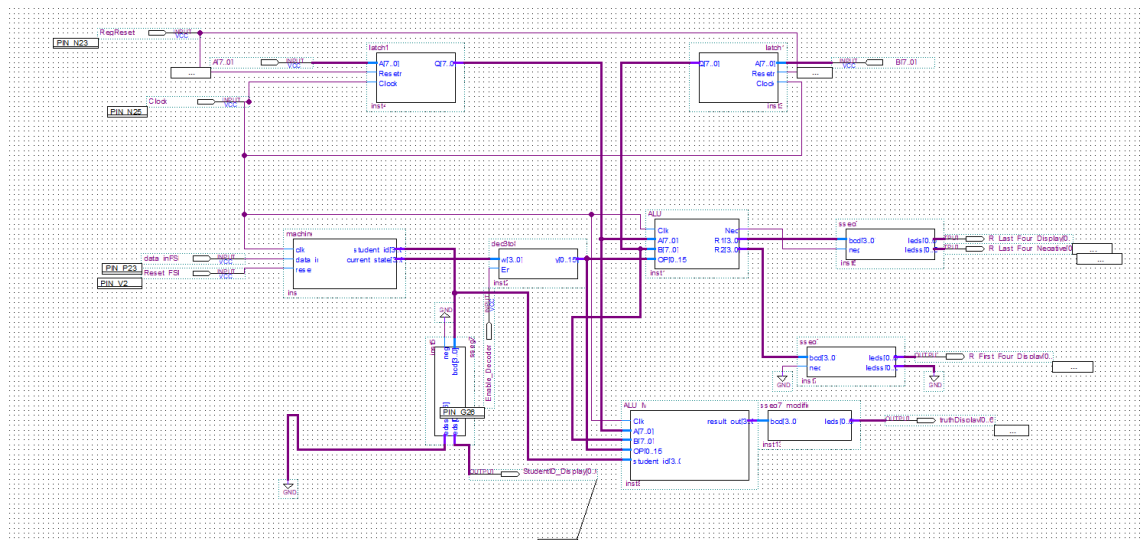


Figure 1: Final Block Diagram that implements all components necessary to create a GPU. Both ALU 1 and ALU 2 are implemented.

Components

Latch 1 & Latch 2

Latch 1 and Latch 2 have similar functionality as they are both registers. However, the only difference between them is their input as they both get fed to the ALU input ports. The last 4 digits of our student number (4679) were taken as hexadecimal: $A = (46)_{16}$ and $B = (79)_{16}$, and converted to an 8-bit binary number. Input A gets fed to Register 1 and input B goes to Register 2. Since the latches are dependent on the clock's rising edge, the input is only read and outputted on the rising edge. It is important to note that the latches can be reset by the user, meaning that the latch will not send A or B to the output.

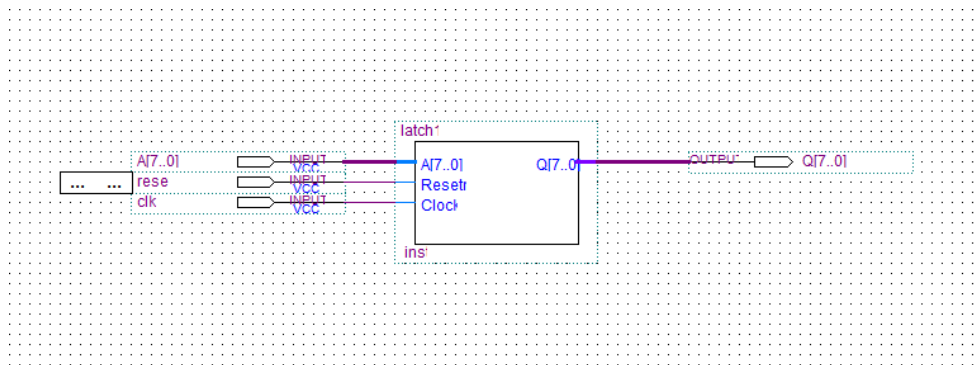


Figure 2:. Block diagram of the Latch component with its respective inputs and outputs connected.

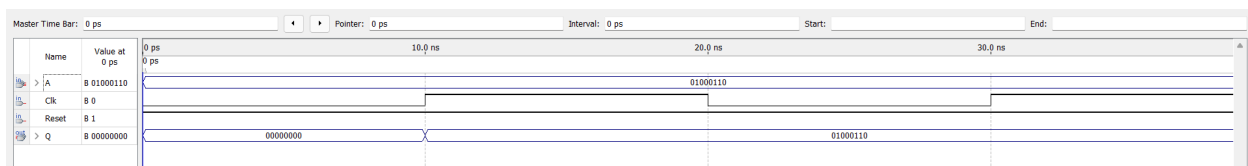


Figure 3:. Waveform for Latch 1 displaying the dependence of the output (Q) on the clock's rising edge.

Table 1*Sample Truth Table for Latch Functionality*

Input		Output
Reset	$A_7 A_6 A_5 A_4 A_3 A_2 A_1 A_0$	$Q_7 Q_6 Q_5 Q_4 Q_3 Q_2 Q_1 Q_0$
0	01000110	00000000
1	01000110	01000110

*only occurs on the rising edge of the clock

4:16 Decoder

The 4:16 decoder works in harmony with the FSM to act as a control unit for the ALU. The decoder takes in an input (w) in binary from the FSM, the current state, and translates that binary number into microcode OP. It is important to note that this only occurs when the enable signal is high. As there are only 9 states, there are only 9 microcodes that will be outputted from the decoder. Reference the table and figures below for further clarification on this component.

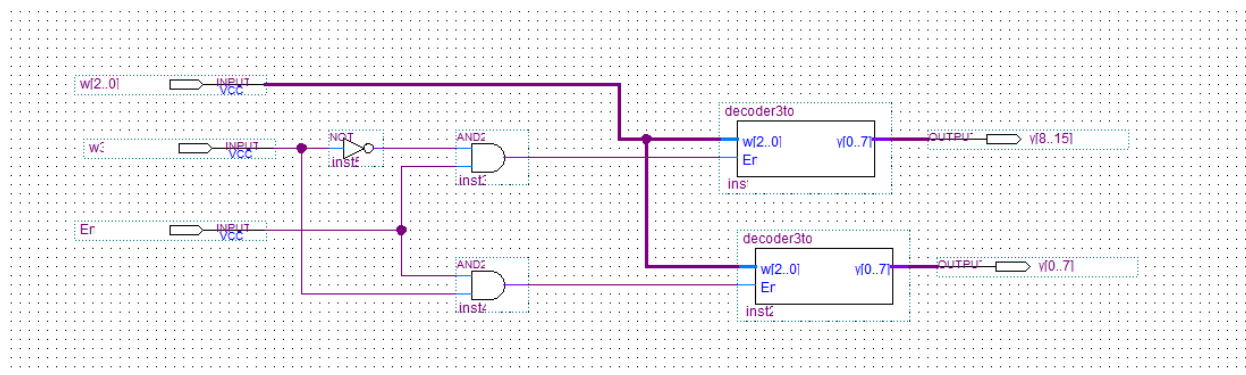


Figure 4:. Block diagram of the 4:16 Decoder component constructed with two 3:8 decoders and logic gates.

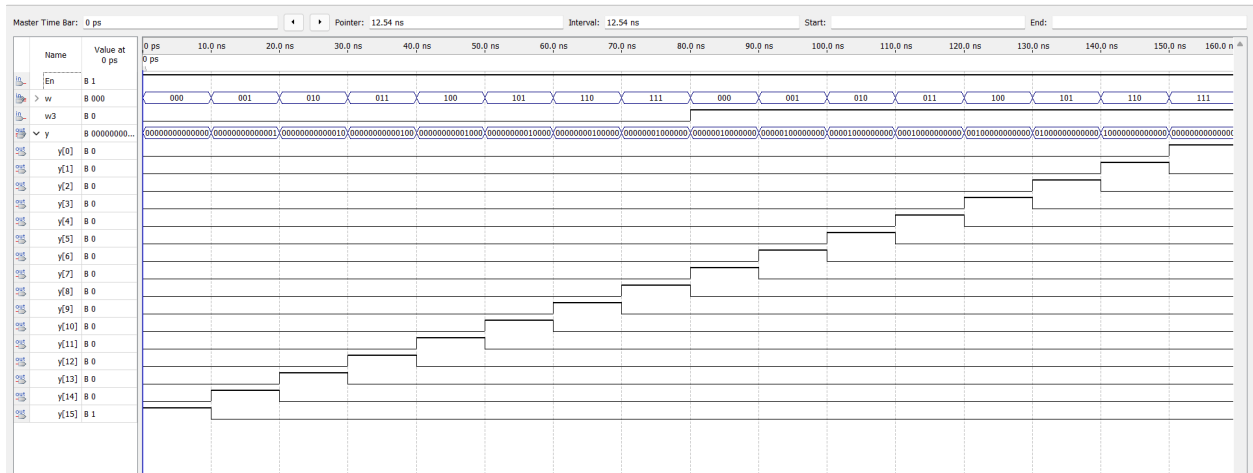


Figure 5:. Waveform for the 4:16 Decoder constructed using two 3:8 Decoders.

Table 2*Truth Table for the 4:16 Decoder*

Input		Output
E_n	w	$y_{15}y_{14}y_{13}y_{12}y_{11}y_{10}y_9y_8y_7y_6y_5y_4y_3y_2y_1y_0$
1	0000	0000000000000001
1	0001	0000000000000010
1	0010	0000000000000100
1	0011	0000000000001000
1	0100	0000000000010000
1	0101	0000000000100000
1	0110	0000000001000000
1	0111	0000000010000000
1	1000	0000000100000000
1	1001	0000001000000000
1	1010	0000010000000000
1	1011	0000100000000000
1	1100	0001000000000000
1	1101	0010000000000000
1	1110	0100000000000000
1	1111	1000000000000000
0	dddd	0000000000000000

Final State Machine (FSM)

The FSM is the second component of the control unit in the GPU. Its role is to dictate the pattern of the unit's sequence. In this case, the machine acts as an up-counter; cycling through states 0 to 8. The current state output will be sent to the decoder, as discussed previously, which will tell the ALU what microcode to run. It is important to note that the FSM used for the GPU is a Moore machine as it is solely dependent on the next state as opposed to a Mealy machine which is dependent on the input and the next state.

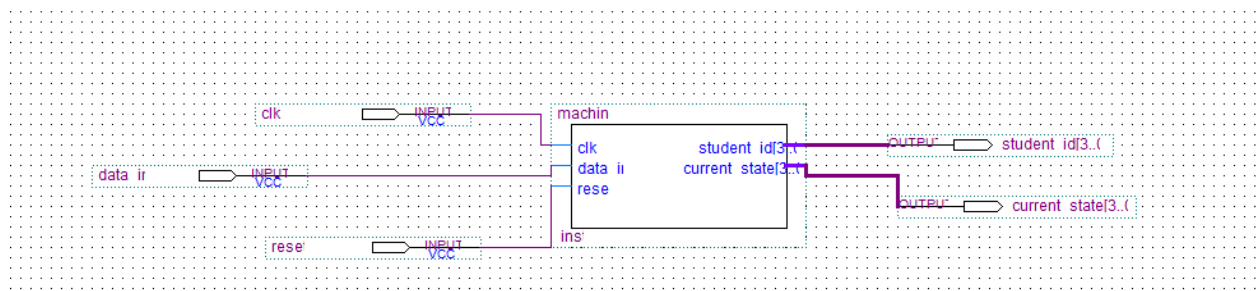


Figure 6:. Block diagram of the FSM component with its inputs and outputs connected.

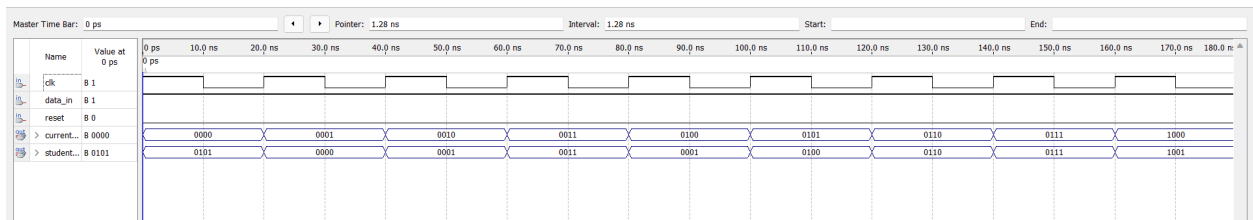


Figure 7:. Waveform for the FSM displaying the relationship between the inputs and the outputs.

Table 3*State Table for the FSM*

Input		Output		
Reset	Data_in	Student_ID	Current_State	Next State
0	1	5	0	1
0	1	0	1	2
0	1	1	2	3
0	1	3	3	4
0	1	1	4	5
0	1	4	5	6
0	1	6	6	7
0	1	7	7	8
0	1	9	8	0

*only occurs on the rising edge of the clock

Problem 1

The purpose of an ALU in general is to process the signals being sent and perform arithmetic and logical operations. In problem 1, the ALU was coded in such a way that it performed a certain function depending on the state of the machine, and evidently, the microcode sent to it. Reference **Table 4** below as it displays the microcode generated by the decoder for ALU_1.

It is important to note the inputs and outputs that are connected to the ALU in order to understand its functionality. The ALU has inputs A[7..0], B[7..0], and OP[0..15]. The two 8-bit numbers are the last 4 digits of the student number as described in the description of Latch 1 & Latch 2 above. The OP signal is the microcode generated by the 4:16 decoder. Using a case statement, each microcode was written to have its own boolean operation as shown in **Table 4**. The microcode results are outputted through 3 outputs: R1[3..0], R2[3..0], and Neg. The result is split into two four bit signals in order to output it to the seven segment displays. Each 4-bit output goes to its display. The negative output goes to the same display as R1 in order to display the [hexadecimal] numbers in order on the FPGA display.

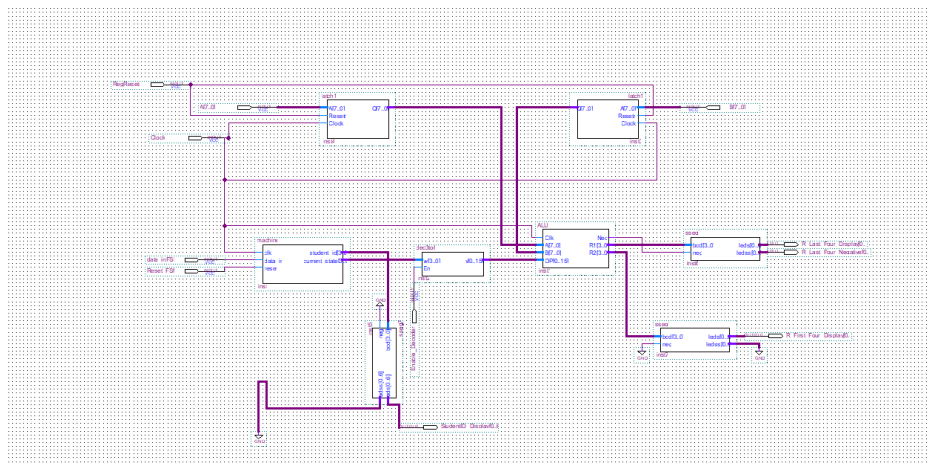


Figure 8: Block Diagram of the GPU implementing ALU_1 and the Seven Segment Display to be shown on the FPGA.

Table 4

Microcodes Generated by the 4:16 Decoder for ALU_1

Input		Output	
E_n	w (Current State)	$y_{15}y_{14}y_{13}y_{12}y_{11}y_{10}y_9y_8y_7y_6y_5y_4y_3y_2y_1y_0$	Boolean Operation
1	0000	0000000000000001	sum(A,B)
1	0001	0000000000000010	diff(A,B)
1	0010	0000000000000100	NOT A
1	0011	0000000000001000	A NAND B
1	0100	0000000000010000	A NOR B
1	0101	0000000000100000	A AND B
1	0110	0000000001000000	A XOR B
1	0111	0000000010000000	A OR B
1	1000	0000000100000000	A XNOR B

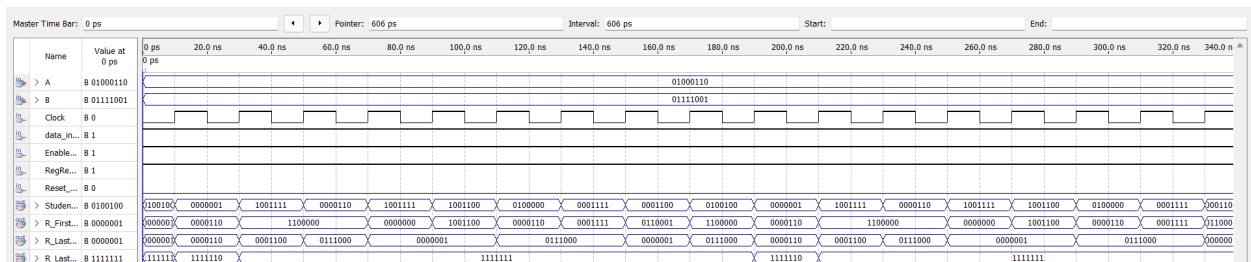


Figure 9:. Waveform for the GPU implementing ALU_1.

Problem 2

Problem 2 entails altering one of the microcodes to compute a different function. I was assigned 4.e: Produce the 2's complement of B. To recall, B in hexadecimal holds the value $(79)_{16}$. Therefore, the two's complement can be determined by NOT-ing all bits of B and adding 1. Resultingly, the output on the FPGA and waveform for microcode 4 must be $(87)_{16}$.

It must be mentioned that ALU_2 has the same input and output signals and connections. The only difference is the output for when the GPU is in the 4th state. Reference **Table 5** to examine the functions and their corresponding states and microcodes.

Table 5

Microcodes Generated by the 4:16 Decoder for ALU_2

Input		Output	
E_n	w (Current State)	$y_{15}y_{14}y_{13}y_{12}y_{11}y_{10}y_9y_8y_7y_6y_5y_4y_3y_2y_1y_0$	Boolean Operation
1	0000	0000000000000001	sum(A,B)
1	0001	0000000000000010	diff(A,B)
1	0010	0000000000000100	NOT A
1	0011	0000000000001000	2's Complement of B
1	0100	0000000000010000	A NOR B
1	0101	0000000000100000	A AND B
1	0110	0000000001000000	A XOR B
1	0111	0000000010000000	A OR B
1	1000	0000000100000000	A XNOR B

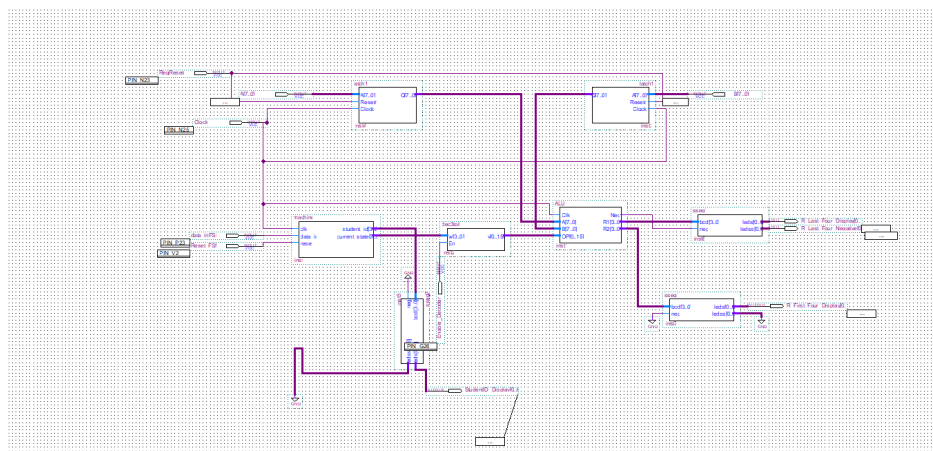


Figure 10:.Block Diagram for the GPU implementing ALU_2.

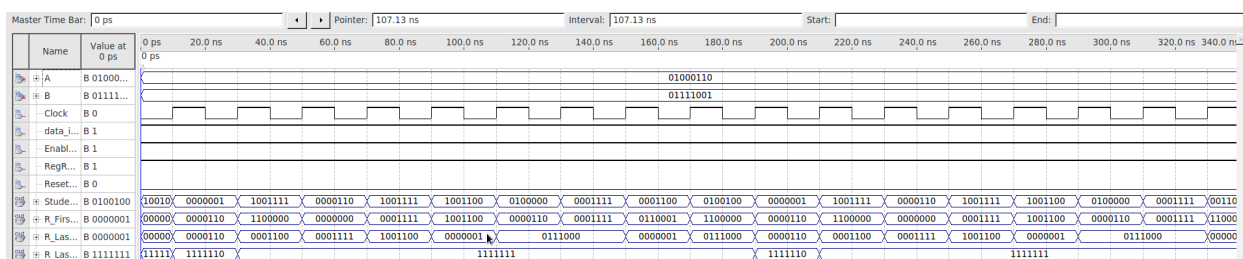


Figure 11:. Waveform for the GPU implementing ALU_2.

Problem 3

The purpose of ALU_3 is to determine if the FSM's output is odd or not. Hence, this component has an extra input signal, `student_Id[3..0]`. The ALU still uses a case statement to perform an operation for each microcode. However, in this case, each microcode does the same operation. Using an if-statement, the least significant bit of the `student_id` signal will be checked. If it is '1', an odd number, the Result output signal will be set to "0000." This output is fed into a modified seven-segment display and prints the letter 'y' on the FPGA. If the LSB is '0', the display will print 'n.'

The GPU design for problem 3 has only one seven segment display, compared to problem 1 & 2 which have two. Additionally, the seven-segment displayed is modified as mentioned previously. It must be noted that ALU_3 has only one 4-bit output signal, Result. The output signal is fed into the modified display. If the 4-bit signal is "0000" the FPGA will display "y." For all other cases, the FPGA will display 'n.'

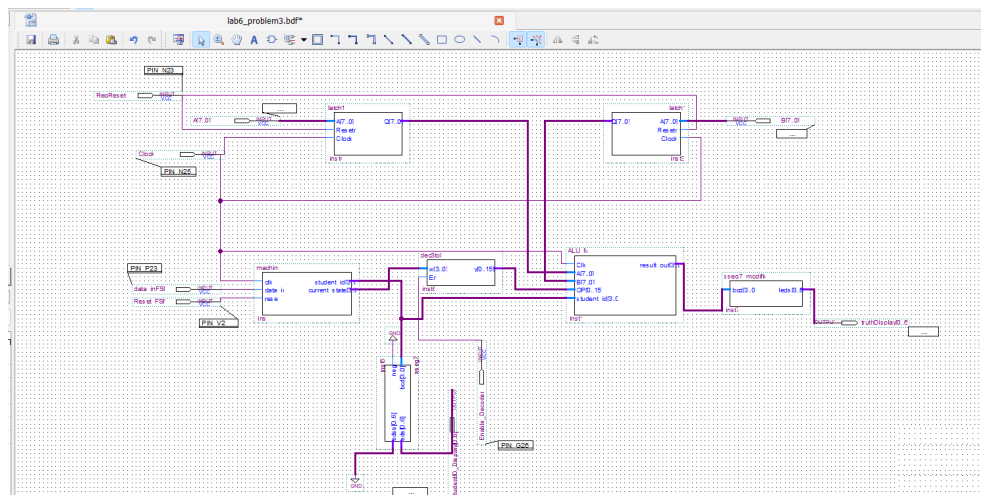
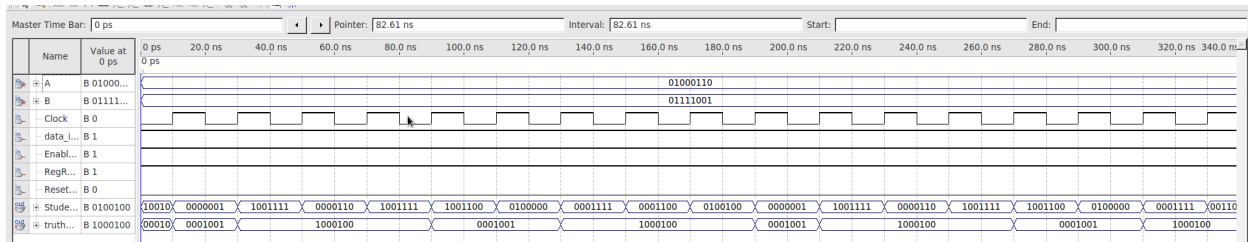


Figure 12:. Block Diagram for the GPU implementing ALU_3.

Table 6*Microcodes Generated by the 4:16 Decoder for ALU_3*

Input		Output
E_n	w (Current State)	$y_{15}y_{14}y_{13}y_{12}y_{11}y_{10}y_9y_8y_7y_6y_5y_4y_3y_2y_1y_0$
1	0000	0000000000000001
1	0001	0000000000000010
1	0010	0000000000000100
1	0011	0000000000001000
1	0100	0000000000010000
1	0101	0000000000100000
1	0110	0000000001000000
1	0111	0000000010000000
1	1000	0000000100000000

**Figure 13:.** Waveform for the GPU implementing ALU_3.

Conclusion

To conclude, each component of the GPU has a purpose in increasing the functionality and performance of its signal processing and computation. From the latch components working as storage units, to the ALU working alongside the control unit to perform the coded functions; each unit is crucial in curating a working GPU.